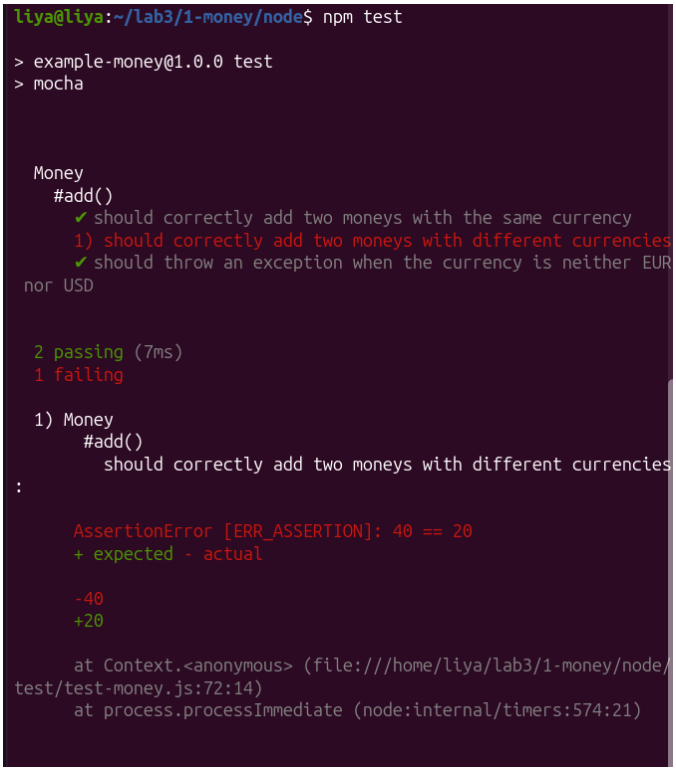
Lab3: Functional and structural unit testing + UI testing

We can observe that the object `m1` is identical in every test. We can try to factorise that:

1. *Open the first example located in `1-money`. You can either work with Mocha in the browser or in Node.JS;*
2. *Create the `m1` variable inside the test suite (after the second `describe('...', function () { // ...`). This variable is now accessible in all three tests;*
3. *Delete the creation of the variable in each test (without deleting variable `m2`);*
4. *Launch again the tests. Understand why the second test no longer works.*



Answer:The second test fails with 40 == 20, actual value 40, expected value 20. Because m1 is now shared by the three tests, the first test has already changed it from 10 to 30 EUR. The second test then adds 20 USD, which is worth 10 EUR, so we get 40 instead of 20.

So far, we only focused on the exercise and verification part of the test. To fix our tests, it is required to reset variable `m1` between each test. To do so, Mocha provides hooks for the setup and teardown parts. Four functions are available: before(), beforeEach(), after(), and afterEach().

1. *Read the documentation about the four functions;*
2. *Identify the function you would need to reset the variable `m1` between each test;*
3. *Refactor the test suite.*

To fix this, I used the beforeEach() hook. It runs before every test and creates a new m1, so each test starts from a new state. After that, all three tests passed again.

```
import assert from 'assert';
import Money from '../src/money.js';

describe('Money', function () {
 let m1;

 // Runs before each test: m1 is recreated, so tests do not affect each
other
 beforeEach(function () {
   m1 = new Money(10.0, 'EUR');
 });

 describe('#add()', function () {
   it('should correctly add two moneys with the same currency', function
() {
     let m2 = new Money(20.0, 'EUR');

     m1.add(m2);

     let newAmount = m1.amount,
       oracle = 30.0;

     assert.equal(newAmount, oracle, `m1 worth ${newAmount}€ while it
should be ${oracle}€`);
   });

   it('should correctly add two moneys with different currencies',
function () {
     let m2 = new Money(20.0, 'USD');

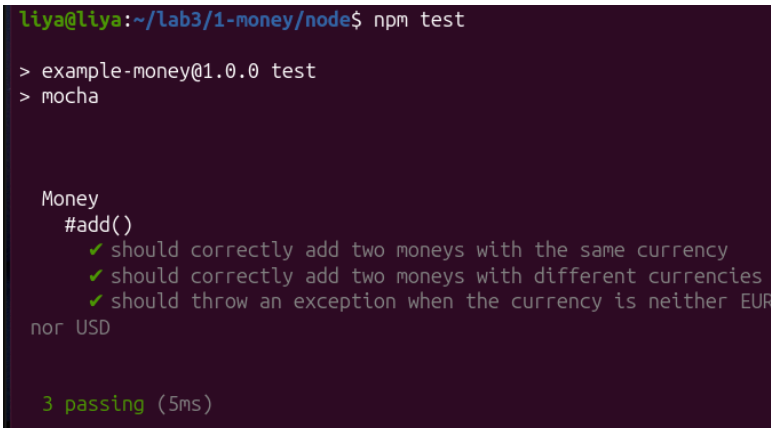
     m1.add(m2);

     let newAmount = m1.amount,
       oracle = 20.0;
```

```
    assert.equal(newAmount, oracle);
  });

  it('should throw an exception when the currency is neither EUR nor
USD', function () {
    let m2 = new Money(20.0, 'BRL');

    assert.throws(function () {
      m1.add(m2);
    });
  });
 });
});
```



So far, we have only used the assert assertion library natively available in Node.JS. It is possible to use others. Chai is one of the most popular ones, proposing three different styles: should(), expect(), and assert(), the latter being similar to the native Node.JS assert library. Below is an example of how the first test of the Money class can be encoded using the two flavours should() and expect():

1. *Install Chai with `npm i --save-dev chai`;*
2. *Use the two flavours to write the two other tests. Use the documentation, especially the part about catching exceptions;*
3. *Understand the differences between `expect` and `should` by reading the documentation.*

With expect, we call a function and give it the value to check: expect(value).to.equal(30).

With should, we first call should() once at the top of the file. Then every object gets a should property, so we can write value.should.equal(30).



*In the following, we will use the TDD approach to design a factorial function which, given a positive integer, returns the factorial value of this number. For example: 4! = 4 * 3 * 2 * 1. Several cases shall be planed:*

- *The given number is a positive integer;*
- *The given number is 0. The returned value is 1;*
- *The given number is a negative integer, undefined, null, or any other value which is not an integer. The program raises an exception.*

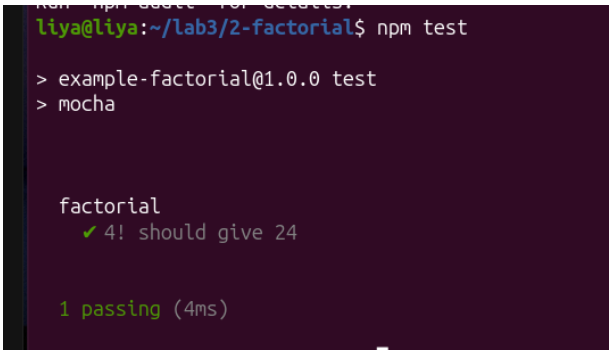
1. *Open the directory `2-factorial` which contains a collection of tests already written;*
2. *Implement in the `src` directory the implementation of the `factorial` function by executing only one test at a time.*

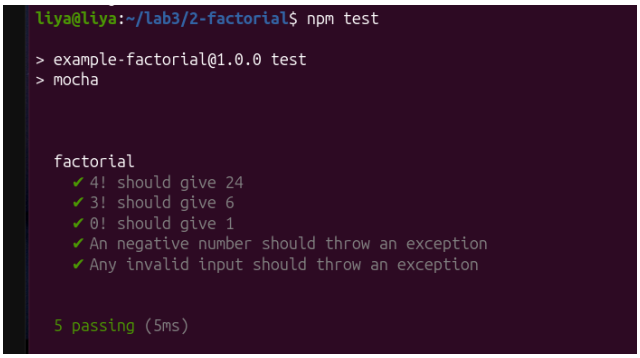
```
/**
* Return the factorial of i.
* @param {number} i - an integer number.
* @return {number} the factorial of i.
*/
export const factorial = function (i) {
```

```
 if (!Number.isInteger(i) || i < 0) {
   throw new Error('factorial expects a non-negative integer');
 }

 let result = 1;
 for (let n = 2; n <= i; n++) {
   result *= n;
 }
 return result;
};

export default factorial;
```





Unit test and code coverage

We will focus on the `src/pawn.js` et `test/pawn.js` files:

1. *Get familiar with the code coverage report on the `src/pawn.js` file. Despite all the existing tests, some instructions and branchings are not covered. Identify them in the report.*
2. *In `test/pawn.js`, observe the tests already written, their structure, and try to write the missing tests such that `pawn.js` are totally covered.*
3. *Have a look at the test starting from line 28 and comment on the label on the test. Is it appropriate?*

The original label in line 28 was "should have a valid movement", but the test checks that two INVALID movements are refused, so the label was misleading.

Problem finding:

After finished pawn.js and game.js, the output is:

```
78 passing (28ms)
8 pending
5 failing

1) Game
     #move and #capture
       black moves from d8 to d6:

    AssertionError [ERR_ASSERTION]: false == true
    + expected - actual

    -false
    +true

    at Context.<anonymous> (file:///home/liya/lab3/3-chess/test/game.js:45:18)
    at process.processImmediate (node:internal/timers:574:21)

2) Game
     #move and #capture
       black moves from d6 to a6:
   AssertionError [ERR_ASSERTION]: there should be a piece at the starting cell
    at Context.<anonymous> (file:///home/liya/lab3/3-chess/test/game.js:41:18)
    at process.processImmediate (node:internal/timers:574:21)

3) Game
     #move and #capture
       black moves from a6 to a2:
   AssertionError [ERR_ASSERTION]: there should be a piece at the starting cell
    at Context.<anonymous> (file:///home/liya/lab3/3-chess/test/game.js:41:18)
    at process.processImmediate (node:internal/timers:574:21)

4) Game
     #move and #capture
       a white piece is captured at a2:

    AssertionError [ERR_ASSERTION]: 'black' == 'white'
    + expected - actual

     black
```

The cause is in src/queen.js: return super.canMove(this, rank, file);. The method Piece.canMove takes only (rank, file), so the extra this shifted the arguments. The parent method received an

object as the rank and returned false, so the queen could never move. I changed the call to super.canMove(rank, file), and all 18 tests of the game passed.

FInal output:

```
  83 passing (29ms)
  8 pending

------------------|---------|----------|---------|---------|-------------------
File              | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
------------------|---------|----------|---------|---------|-------------------
All files         |   96.62 |    85.71 |   88.88 |   96.62 |
 src              |   97.06 |     89.1 |    87.5 |   97.06 |
  bishop.js       |     100 |       75 |     100 |     100 | 33
  chessboard.js   |     100 |      100 |     100 |     100 |
  color.js        |     100 |      100 |     100 |     100 |
  king.js         |   76.31 |      100 |      50 |   76.31 | 27-35
  knight.js       |   95.55 |    83.33 |     100 |   95.55 | 40-41
  pawn.js         |     100 |      100 |     100 |     100 |
  piece.js        |   98.01 |    84.61 |      75 |   98.01 | 139-140,150-152
  queen.js        |   95.55 |    85.71 |     100 |   95.55 | 40-41
  rook.js         |   95.45 |       60 |     100 |   95.45 | 39-40
 src/notifications |   93.25 |    54.54 |     100 |   93.25 |
  capture.js      |   93.33 |       50 |     100 |   93.33 | 24-25
  move.js         |   89.74 |       40 |     100 |   89.74 | 24-25,33-34
  notification.js |     100 |      100 |     100 |     100 |
------------------|---------|----------|---------|---------|-------------------
```

pawn.js

```
import assert from 'assert';
import Color from '../src/color.js';
import Chessboard from '../src/chessboard.js';
import Pawn from '../src/pawn.js';

describe('White pawn', function () {
 describe('#canMove() from its initial position', function () {
   let chessboard, pawn;

   beforeEach(function () {
     chessboard = new Chessboard();
     pawn = new Pawn({
       chessboard: chessboard,
       color: Color.WHITE,
       rank: 2,
       file: 3
     });
     chessboard.pieces.push(pawn);
   });
```

```
  it('should be able to move forward 1 rank', function () {
    const output = pawn.canMove(pawn.rank + 1, pawn.file),
      oracle = true;

    assert.equal(output, oracle);
  });

  // The original label was "should have a valid movement", but the test
  // checks that two INVALID movements are refused, so the label was
misleading.
  it('should not be able to move to an invalid position', function () {
    assert.equal(pawn.canMove(pawn.rank + 2, pawn.file + 1), false);
    assert.equal(pawn.canMove(pawn.rank - 10, pawn.file), false);
  });

  it('should not be able to move in diagonal (when not capturing)',
function () {
    const output = pawn.canMove(pawn.rank + 1, pawn.file + 1),
      oracle = false;

    assert.equal(output, oracle);
  });

  it('should be able to move in diagonal when capturing', function () {
    const blackPawn = new Pawn({
      chessboard: chessboard,
      color: Color.BLACK,
      rank: 3,
      file: 4
    });
    chessboard.pieces.push(blackPawn);

    const output = pawn.canMove(pawn.rank + 1, pawn.file + 1),
      oracle = true;

    assert.equal(output, oracle);
  });
```

```
  it('should be able to move forward 2 ranks from its initial position',
function () {
    const output = pawn.canMove(pawn.rank + 2, pawn.file),
      oracle = true;

    assert.equal(output, oracle);
  });

  it('should not be able to move backward 1 rank', function () {
    const output = pawn.canMove(pawn.rank - 1, pawn.file),
      oracle = false;

    assert.equal(output, oracle);
  });
});

describe('#move() from its initial position', function () {
  let chessboard, pawn, rank, file;

  beforeEach(function () {
    chessboard = new Chessboard();
    pawn = new Pawn({
      chessboard: chessboard,
      color: Color.WHITE,
      rank: 2,
      file: 3
    });

    chessboard.pieces.push(pawn);

    rank = pawn.rank;
    file = pawn.file;
  });

  it('should be able to move forward 1 rank', function () {
    const oracle = { rank: rank + 1, file: file };

    pawn.move(rank + 1, file);

    assert.equal(pawn.rank, oracle.rank);
```

```
    assert.equal(pawn.file, oracle.file);
  });

  it('should not be able to move in diagonal (when not capturing)',
function () {
    const oracle = { rank: rank, file: file };

    pawn.move(rank + 1, file + 1);

    assert.equal(pawn.rank, oracle.rank);
    assert.equal(pawn.file, oracle.file);
  });

  it('should be able to move forward 2 ranks from its initial position',
function () {
    const oracle = { rank: rank + 2, file: file };

    pawn.move(rank + 2, file);

    assert.equal(pawn.rank, oracle.rank);
    assert.equal(pawn.file, oracle.file);
  });

  it('should not be able to move backward 1 rank', function () {
    const oracle = { rank: rank, file: file };

    pawn.move(rank - 1, file);

    assert.equal(pawn.rank, oracle.rank);
    assert.equal(pawn.file, oracle.file);
  });
});

describe('#canMove() from a non-initial position', function () {
  let chessboard, pawn;

  beforeEach(function () {
    chessboard = new Chessboard();
    pawn = new Pawn({
      chessboard: chessboard,
```

```
      color: Color.WHITE,
      rank: 3,
      file: 3
    });
  });

  it('should not be able to move forward 2 ranks from a non-initial
position', function () {
    const output = pawn.canMove(pawn.rank + 2, pawn.file),
      oracle = false;

    assert.equal(output, oracle);
  });
 });
});

describe('Black pawn', function () {
 describe('#canMove() from its initial position', function () {
  let chessboard, pawn;

  beforeEach(function () {
    chessboard = new Chessboard();
    pawn = new Pawn({
      chessboard: chessboard,
      color: Color.BLACK,
      rank: 7,
      file: 3
    });
  });

  it('should be able to move forward 1 rank (decreasing)', function () {
    const output = pawn.canMove(pawn.rank - 1, pawn.file),
      oracle = true;

    assert.equal(output, oracle);
  });

  it('should be able to move forward 2 ranks from its initial position',
function () {
    const output = pawn.canMove(pawn.rank - 2, pawn.file),
```

```
      oracle = true;

    assert.equal(output, oracle);
  });

  it('should not be able to move backward 1 rank (increasing)', function
() {
    const output = pawn.canMove(pawn.rank + 1, pawn.file),
      oracle = false;

    assert.equal(output, oracle);
  });

  it('should not be able to move forward 3 ranks', function () {
    const output = pawn.canMove(pawn.rank - 3, pawn.file),
      oracle = false;

    assert.equal(output, oracle);
  });

  it('should not be able to move 2 files sideways', function () {
    const output = pawn.canMove(pawn.rank - 1, pawn.file + 2),
      oracle = false;

    assert.equal(output, oracle);
  });

  it('should be able to move in diagonal when capturing', function () {
    const whitePawn = new Pawn({
      chessboard: chessboard,
      color: Color.WHITE,
      rank: 6,
      file: 4
    });
    chessboard.pieces.push(whitePawn);

    const output = pawn.canMove(pawn.rank - 1, pawn.file + 1),
      oracle = true;

    assert.equal(output, oracle);
```

```
    });
  });
});
```

game.js

```
import * as game from '../games/2022-03-07-game.json' with { type: 'json'
};
import assert from 'assert';
import Color from '../src/color.js';
import Chessboard from '../src/chessboard.js';

/**
* From a file number, get the corresponding letter.
* Rank are 1-indexed.
* Letters are 97-indexed to get lower-alpha characters.
* @param {number} file - the file in its number representation.
* @return {string} returns the file in its string representation.
*/
const convertFileToString = function (file) {
  return String.fromCharCode(97 - 1 + file);
};

describe('Game', function () {
  let chessboard;

  let lastCaptured = null;

  before(function () {
    chessboard = new Chessboard();
    chessboard.init();
  });

  describe('#move and #capture', function () {
    game.default.forEach((action) => {
      const color = Color[action.color.toUpperCase()];

      if (action.type === 'move') {
        const { from, to } = action;
```

```
      it(`${action.color} moves from
${convertFileToString(from.file)}${from.rank} to
${convertFileToString(to.file)}${to.rank}`, function () {
        const piece = chessboard.getPiece(from.rank, from.file);
        assert.ok(piece, 'there should be a piece at the starting cell');
        assert.equal(piece.color, color);

        assert.equal(piece.canMove(to.rank, to.file), true);


        lastCaptured = chessboard.getPiece(to.rank, to.file);
        piece.move(to.rank, to.file);

        assert.equal(piece.rank, to.rank);
        assert.equal(piece.file, to.file);
        assert.equal(chessboard.getPiece(to.rank, to.file), piece);
        assert.equal(chessboard.getPiece(from.rank, from.file), null);
      });
    } else {
      const { at } = action;

      it(`a ${action.color} piece is captured at
${convertFileToString(at.file)}${at.rank}`, function () {
        assert.ok(lastCaptured, 'the previous move should have captured a
piece');

        assert.equal(lastCaptured.color, color);

        assert.equal(lastCaptured.isCaptured, true);
        assert.equal(chessboard.pieces.includes(lastCaptured), false);

        const occupant = chessboard.getPiece(at.rank, at.file);
        assert.ok(occupant);
        assert.notEqual(occupant.color, color);
      });
    }
  });
 });
});
```